ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN

BÁO CÁO CUỐI KỲ

MÔN: HỌC MÁY TIÊN TIẾN

ĐỀ TÀI

HỆ HỎI ĐÁP LỊCH SỬ TIẾNG VIỆT DỰA TRÊN TRUY HỒI VÀ MÔ HÌNH SINH VIT5

Nhóm Học viên thực hiện: Nhóm 10
1. Nguyễn Duy Phong - 240201021
2. Hoàng Hải Đăng - 2502010484
3. Trần Anh Hào - 250201058

Giảng viên hướng dẫn: TS. Hà Minh Tân

TP. Hồ Chí Minh, tháng 11 năm 2026

MỤC LỤC

1. GIỚI THIỆU 6
1.1. Bối cảnh và động lực 6
1.2. Phát biểu bài toán 6
1.3. Câu hỏi nghiên cứu 6
1.4. Đóng góp chính 7
2. CƠ SỞ LÝ THUYẾT VÀ NGHIÊN CỨU LIÊN QUAN 8
2.1. Đọc hiểu máy và câu hỏi không trả lời được 8
2.2. Hỏi đáp miền mở theo kiến trúc Retriever–Reader 8
2.3. Truy hồi thưa, truy hồi dày và gộp thứ hạng 8
2.4. Mô hình sinh T5/ViT5 và mô hình trích xuất 8
3. DỮ LIỆU 9
3.1. Nguồn dữ liệu và kho tri thức 9
3.2. Chia train / dev / test theo bài 9
3.3. Làm sạch dữ liệu 9
3.4. Thống kê độ dài và lựa chọn tham số 10
4. PHƯƠNG PHÁP 11
4.1. Retriever hybrid 11
4.2. Reader ViT5 11
4.3. Ngưỡng từ chối 12
4.4. Các baseline 12
4.5. Thước đo và kiểm định 13
5. THỰC NGHIỆM VÀ KẾT QUẢ 14
5.1. Thiết lập 14
5.2. Retriever (E2) 14
5.3. Reader đọc đoạn văn đúng (E1, E4, E5) 15
5.4. Toàn hệ thống (E2, E3) 16
5.5. Ngưỡng từ chối 17
5.6. Theo miền và loại câu hỏi (E6) 17
5.7. Độ ổn định 18
5.8. So với mục tiêu đặt ra 19
6. PHÂN TÍCH LỖI 20
7. DEMO VÀ ĐỘ TRỄ 22
7.1. Giao diện demo 22
7.2. Độ trễ 23
8. KẾT LUẬN, HẠN CHẾ VÀ HƯỚNG PHÁT TRIỂN 24
8.1. Kết luận 24
8.2. Hạn chế 24
8.3. Hướng phát triển 24
TÀI LIỆU THAM KHẢO 25

DANH MỤC HÌNH ẢNH

Số hiệu	Tên hình
Hình 1.1	Kiến trúc hệ hỏi đáp: Retriever hybrid tìm top-k đoạn, Reader ViT5 đọc từng đoạn và chọn đáp án
Hình 5.1	Recall@k trên dev của BM25, dense và hybrid đã chọn
Hình 5.2	F1 trên dev của các Reader khi đọc đoạn đúng
Hình 5.3	F1 toàn hệ thống trên test theo k
Hình 5.4	F1 trên test theo miền
Hình 5.5	F1 theo loại câu hỏi của hệ thống cuối trên test
Hình 6.1	Phân bố các nhóm lỗi trong 100 câu sai
Hình 7.1	Demo Gradio: câu hỏi về chồng của Mary Stuart, đáp án François II được tô đậm trong đoạn bằng chứng

DANH MỤC BẢNG BIỂU

Số hiệu	Tên bảng
Bảng 1.1	Các thí nghiệm và câu hỏi nghiên cứu tương ứng
Bảng 3.1	Phân chia dữ liệu theo bài (VN/TG: số câu hỏi về lịch sử Việt Nam / thế giới)
Bảng 3.2	Các lỗi dữ liệu đã phát hiện và sửa
Bảng 3.3	Độ dài theo token ViT5 (nguồn: reports/data_stats.md)
Bảng 4.1	Cấu hình huấn luyện Reader ViT5 (R2 chính)
Bảng 4.2	Các baseline
Bảng 4.3	Chuẩn hoá trước khi tính EM/F1
Bảng 5.1	Recall@k và MRR của retriever trên dev (892 câu trả lời được)
Bảng 5.2	Reader khi được cho sẵn đoạn văn đúng (greedy)
Bảng 5.3	F1 toàn hệ thống trên test theo retriever và số đoạn k (Reader R2 chính, trừ dòng cuối)
Bảng 5.4	F1 theo miền trên test
Bảng 5.5	So sánh với các chỉ số mục tiêu đặt ra khi lập kế hoạch
Bảng 6.1	Phân loại 100 câu sai của hệ thống cuối trên test
Bảng 7.1	Độ trễ mỗi câu hỏi (ms, trung vị; câu hỏi ngẫu nhiên từ dev)

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Ý nghĩa
BM25	Best Matching 25 – hàm xếp hạng tìm kiếm theo từ khoá
CI	Confidence Interval – khoảng tin cậy
Dense	Truy hồi dày, dựa trên vector ngữ nghĩa
EM	Exact Match – khớp hoàn toàn
F1	Trung bình điều hoà của precision và recall theo âm tiết
FAISS	Facebook AI Similarity Search – thư viện tìm kiếm vector
FiD	Fusion-in-Decoder
HasAns / NoAns	Nhóm câu trả lời được / không trả lời được
MRC	Machine Reading Comprehension – đọc hiểu máy
MRR	Mean Reciprocal Rank
QA	Question Answering – hỏi đáp
R@k	Recall@k – tỉ lệ câu hỏi có đoạn đúng trong top-k
RAG	Retrieval-Augmented Generation
RQ	Research Question – câu hỏi nghiên cứu
RRF	Reciprocal Rank Fusion – gộp thứ hạng nghịch đảo
TG / VN	Lịch sử thế giới / lịch sử Việt Nam
ViQuAD	Vietnamese Question Answering Dataset
ViT5	Mô hình T5 huấn luyện trước cho tiếng Việt

BẢNG PHÂN CÔNG CÔNG VIỆC

STT	Thành viên	Vai trò	Nội dung công việc chi tiết	Đóng góp
1	**Nguyên Duy Phong - 240201021**	**Nhóm trưởng**	**[GĐ0 – Dọn dẹp và bảo mật]** • Đánh giá dự án cũ, thu hồi API key lộ, dựng lại repo, ghim requirements.txt **[GĐ1 – Dữ liệu]** • Rà soát và sửa 5 nhóm lỗi dữ liệu (1.988 câu không nhãn, plausible_answer, id trùng, lệch vị trí) • Chia train/dev/test theo bài, test chống rò rỉ; thống kê độ dài token **[GĐ2 – Retriever]** • BM25 âm tiết/tách từ, dense bi-encoder + FAISS, RRF có trọng số; quét tham số trên dev **[Báo cáo]** • Viết chương 2–3, slide trình bày	**32,5%**
2	**Hoàng Hải Đăng - 2502010484**	**Thành viên**	**[GĐ3 – Thước đo và baseline]** • Metric EM/F1 tiếng Việt kiểu SQuAD 2.0, định dạng dự đoán chung, công cụ chấm • Baseline B0 (ngẫu nhiên), B1 (XLM-R trích xuất, Colab), B2 (ViT5 zero-shot) **[GĐ6 – Demo và độ trễ]** • Pipeline dùng chung, demo Gradio, đo độ trễ CPU/GPU **[GĐ7 – Tổng hợp]** • results.md, biểu đồ, README tái lập **[Báo cáo]** • Viết chương 4 và 7	**32,5%**
3	**Trần Anh Hào - 250201058**	**Thành viên**	**[GĐ4 – Reader ViT5]** • Dữ liệu R1/R2 (đoạn nhiễu), huấn luyện ViT5 trên Colab và GPU local, sửa lỗi đánh giá dev **[GĐ5 – Thí nghiệm E1–E6]** • Dò tham số, ngưỡng từ chối, E4/E5, 3 seed, bootstrap CI, chạy test một lần **[GĐ6 – Phân tích lỗi]** • Phân loại 100 câu sai, hậu xử lý ký tự ngoài từ vựng **[Báo cáo]** • Viết chương 1, 5, 6, 8; tổng hợp báo cáo	**35%**
			TỔNG CỘNG	**100%**

1. GIỚI THIỆU

1.1. Bối cảnh và động lực

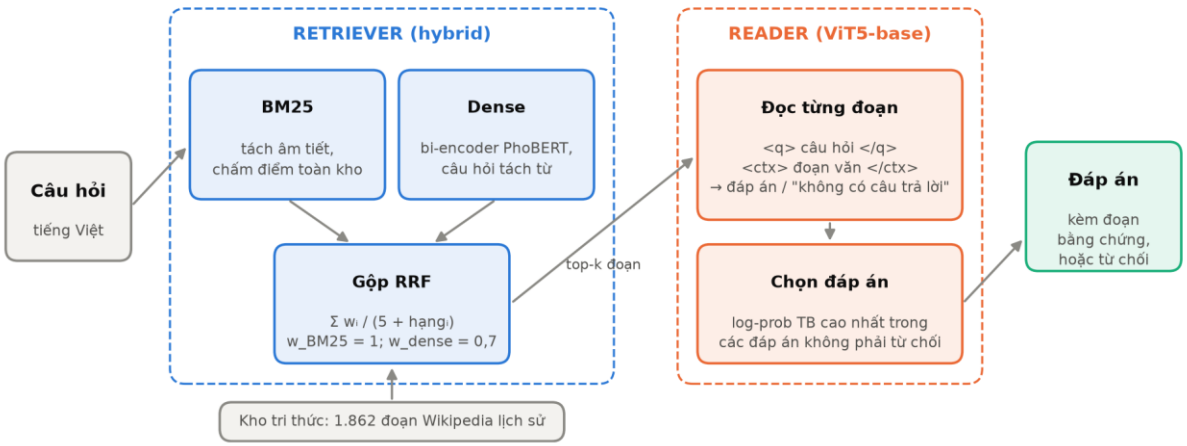
Người học và người tra cứu lịch sử thường đặt những câu hỏi cụ thể: một sự kiện xảy ra khi nào, ai lãnh đạo, vì sao một triều đại suy yếu. Câu trả lời thường nằm sẵn trong các bài viết bách khoa như Wikipedia, nhưng rải rác trong hàng nghìn đoạn văn. Một hệ thống hỏi đáp tự động cần làm được hai việc: tìm đúng đoạn văn có thông tin, rồi rút ra câu trả lời ngắn gọn từ đoạn đó. Ngoài ra, hệ thống phải biết **từ chối** khi tài liệu không chứa câu trả lời, thay vì đưa ra một đáp án nghe hợp lý nhưng sai.

Với tiếng Việt, bộ dữ liệu UIT-ViQuAD 2.0 [2] cung cấp câu hỏi trên Wikipedia tiếng Việt, gồm cả câu hỏi *không trả lời được*. Mô hình ViT5 [7] là mô hình sinh văn bản được huấn luyện trước trên tiếng Việt. Đề tài kết hợp hai nguồn lực này để xây dựng một hệ hỏi đáp **miền mở** cho lĩnh vực lịch sử: người dùng chỉ nhập câu hỏi, hệ thống tự tìm tài liệu và trả lời.

1.2. Phát biểu bài toán

Đầu vào: một câu hỏi lịch sử bằng tiếng Việt. **Tài liệu:** kho tri thức gồm 1.862 đoạn văn từ 38 bài Wikipedia lịch sử. **Đầu ra:** một câu trả lời ngắn kèm đoạn văn làm bằng chứng, hoặc câu "không có câu trả lời" khi kho không chứa thông tin cần thiết.

Hệ thống gồm hai khâu theo kiến trúc Retriever–Reader [3] (Hình 1.1). **Retriever** chấm điểm toàn bộ kho và lấy k đoạn liên quan nhất, kết hợp tìm theo từ khoá (BM25) và tìm theo ngữ nghĩa (bi-encoder). **Reader** là ViT5 đã fine-tune, đọc từng đoạn và *sinh* ra câu trả lời. Khác với mô hình trích xuất, Reader không bị buộc phải chọn một đoạn chữ có sẵn, nên học được cách nói "không có câu trả lời" như một đầu ra bình thường.



Hình 1.1: Kiến trúc hệ hỏi đáp: Retriever hybrid tìm top-k đoạn, Reader ViT5 đọc từng đoạn và chọn đáp án

1.3. Câu hỏi nghiên cứu

- **RQ1:** Reader sinh đáp án (ViT5) so với Reader trích xuất (XLM-R) thì bên nào tốt hơn trên dữ liệu lịch sử tiếng Việt?

- **RQ2:** Truy hồi kết hợp (BM25 + dense) có tốt hơn từng phương pháp riêng không? Sai số của toàn hệ thống đến từ khâu tìm hay khâu đọc?
- **RQ3:** Huấn luyện với câu không trả lời được có giúp mô hình biết từ chối thay vì bịa không?
- **RQ4:** Thêm dữ liệu ViQuAD ngoài miền lịch sử có giúp ích không? Hiệu năng trên bài lịch sử Việt Nam và lịch sử thế giới khác nhau thế nào?

Sáu thí nghiệm E1–E6 trả lời các câu hỏi trên (Bảng 1.1). Mọi lựa chọn cấu hình đều làm trên tập dev; tập test chỉ được chấm **một lần** cho các cấu hình cuối cùng.

Mã	Thí nghiệm	Trả lời
E1	Reader trích xuất (B1) so với ViT5 sinh (R1/R2), đọc trên đoạn văn đúng	RQ1
E2	Toàn hệ thống: retriever {BM25, dense, hybrid} × số đoạn k {1, 3, 5}	RQ2
E3	Đọc đoạn đúng so với đoạn tự tìm, để tách sai số tìm và sai số đọc	RQ2
E4	Huấn luyện có so với không có câu không trả lời được	RQ3
E5	Thêm 10.000 câu ViQuAD ngoài miền lịch sử khi huấn luyện	RQ4
E6	Tách kết quả theo miền (Việt Nam / thế giới) và loại câu hỏi	RQ4

Bảng 1.1: Các thí nghiệm và câu hỏi nghiên cứu tương ứng

1.4. Đóng góp chính

- Một kho dữ liệu hỏi đáp lịch sử sạch, chia train/dev/test **theo bài** để chống rò rỉ, đã sửa 5 nhóm lỗi dữ liệu kế thừa (mục 3.3).
- Retriever hybrid có trọng số, đưa đoạn đúng vào top-5 ở 93,4% câu trên dev và 90,2% trên test.
- Reader ViT5 được huấn luyện thêm với đoạn văn nhiễu do retriever tìm ra. Reader này vượt mô hình trích xuất XLM-R chủ yếu nhờ khả năng từ chối.
- Bộ thí nghiệm có đối chứng: cùng thước đo, cùng định dạng dự đoán, khoảng tin cậy bootstrap, ba seed. Kèm phân tích tay 100 câu sai, demo Gradio và số đo độ trễ.
- Mã nguồn tái lập được: phiên bản thư viện được ghim, mỗi file dự đoán ghi commit git, có unit test, README hướng dẫn tái lập từng bảng.

Mã nguồn: https://github.com/trananhhao-0603/hoc_may_tien_tien (mã, dữ liệu đã xử lý, toàn bộ bảng kết quả và lịch sử commit; checkpoint mô hình không đưa lên vì dung lượng lớn, có thể huấn luyện lại theo README).

2. CƠ SỞ LÝ THUYẾT VÀ NGHIÊN CỨU LIÊN QUAN

2.1. Đọc hiểu máy và câu hỏi không trả lời được

Bài toán đọc hiểu máy (MRC) cho mô hình một đoạn văn và một câu hỏi, yêu cầu chỉ ra đoạn chữ trả lời. SQuAD 2.0 [1] bổ sung câu hỏi *không trả lời được*: các câu này được viết sao cho nghe hợp lý và gần với đoạn văn, nhưng đoạn văn không chứa đáp án. Mô hình phải nhận ra để từ chối. UIT-ViQuAD 2.0 [2] áp dụng cách làm này cho tiếng Việt và kèm một *đáp án mồi* (plausible answer) cho mỗi câu không trả lời được.

2.2. Hỏi đáp miền mở theo kiến trúc Retriever–Reader

DrQA [3] là hệ đầu tiên kết hợp một bộ tìm kiếm TF-IDF trên Wikipedia với một mô hình đọc hiểu. DPR [4] thay tìm kiếm từ khoá bằng tìm kiếm vector với hai bộ mã hoá BERT. RAG [5] và Fusion-in-Decoder (FiD) [6] dùng mô hình sinh làm Reader: RAG sinh câu trả lời dựa trên các đoạn tìm được, còn FiD mã hoá riêng từng đoạn rồi để bộ giải mã đọc tất cả cùng lúc. Đề tài dùng cách đơn giản hơn FiD: đọc từng đoạn độc lập rồi chọn đáp án theo điểm tin cậy. Lý do là bộ nhớ GPU có hạn và trung vị một đoạn văn đã khoảng 200 token.

2.3. Truy hồi thưa, truy hồi dày và gộp thứ hạng

BM25 [9] chấm điểm theo các từ trùng giữa câu hỏi và đoạn văn, có trọng số theo độ hiếm của từ. Cách này rất mạnh với tên riêng và năm, vốn dày đặc trong văn bản lịch sử. Truy hồi dày mã hoá câu hỏi và đoạn văn thành vector rồi so độ gần, hiểu được cách diễn đạt khác nhau. Đề tài dùng mô hình vietnamese-bi-encoder [14] dựa trên PhoBERT [12] và thư viện FAISS [13]. Reciprocal Rank Fusion (RRF) [10] gộp hai danh sách theo *thứ hạng*: điểm = $\Sigma\ w_i / (k + \text{hạng}_i)$. Nhờ vậy không cần chuẩn hoá hai thang điểm khác nhau.

2.4. Mô hình sinh T5/ViT5 và mô hình trích xuất

T5 [8] đưa mọi bài toán xử lý ngôn ngữ về dạng văn bản → văn bản. ViT5 [7] là T5 được huấn luyện trước trên kho văn bản tiếng Việt lớn; đề tài dùng bản base. Đối chứng là mô hình *trích xuất* XLM-RoBERTa [11]: mô hình dự đoán vị trí bắt đầu và kết thúc của đáp án, và trỏ vào token đặc biệt đầu chuỗi khi từ chối. Mô hình trích xuất không thể bịa, nhưng phải cắt đúng ranh giới đáp án. Mô hình sinh linh hoạt hơn, nhưng có nguy cơ sinh ra nội dung không có trong văn bản. Đề tài đo trực tiếp nguy cơ này (mục 5.4, 6).

3. DỮ LIỆU

3.1. Nguồn dữ liệu và kho tri thức

Dữ liệu lấy từ UIT-ViQuAD 2.0 trên HuggingFace (taidng/UIT-ViQuAD2.0). Nhóm khảo sát toàn bộ bài viết, rà tay để đánh dấu bài thuộc lĩnh vực lịch sử và bài về Việt Nam, rồi giữ lại **38 bài** (9 bài lịch sử Việt Nam, 29 bài lịch sử thế giới). Kho tri thức gồm **1.862 đoạn văn**. Tập câu hỏi có nhãn gồm **12.009 câu**: 8.126 câu trả lời được và 3.883 câu không trả lời được (32,3%).

3.2. Chia train / dev / test theo bài

Các câu hỏi trong cùng một bài thường hỏi về cùng các sự kiện. Nếu chia ngẫu nhiên theo câu, mô hình sẽ "thấy trước" nội dung của tập test và điểm bị thổi phồng. Vì vậy dữ liệu được chia **theo bài**, cân bằng số đoạn khoảng 80/10/10 (Bảng 3.1). Kho tri thức chứa đoạn văn của cả ba tập. Điều này hợp lệ vì kho là tài liệu hệ thống được phép tra cứu; thứ cần giấu là các cặp hỏi–đáp của tập test.

Tập	Số bài	Số đoạn	Trả lời được	Không trả lời được	Tổng câu hỏi	VN / TG
train	23	1.491	6.347	3.047	9.394	2.610 / 6.784
dev	8	179	892	437	1.329	407 / 922
test	7	192	887	399	1.286	420 / 866

Bảng 3.1: Phân chia dữ liệu theo bài (VN/TG: số câu hỏi về lịch sử Việt Nam / thế giới)

Test gồm các bài Mahatma Gandhi, Elizabeth I của Anh, Hồ Quý Ly, Nhà Trần, Louis XVI của Pháp và Đế quốc La Mã Thần thánh. Unit test `test_split_leak.py` kiểm tra trên dữ liệu thật rằng không có bài, đoạn hay câu hỏi nào nằm ở hai tập.

3.3. Làm sạch dữ liệu

Dữ liệu kế thừa từ dự án trước có nhiều lỗi. Nhóm đã tìm ra và sửa các lỗi trong Bảng 3.2 trước khi huấn luyện.

Lỗi phát hiện	Nguyên nhân	Cách xử lý
1.988 câu "trả lời được" nhưng đáp án không có trong đoạn văn	Split test chính thức của ViQuAD không có nhãn (đáp án rỗng) nhưng bị coi là có đáp án	Bỏ các câu không nhãn, vẫn giữ đoạn văn trong kho
Trường plausible_answer rỗng ở mọi dòng	Đọc nhầm tên trường plausible thay vì plausible_answers	Sửa lại, đủ 3.883/3.883 câu
152 id câu hỏi trùng giữa train và validation gốc	Hai split gốc dùng chung dạng id	Đổi thành dạng train:0013-0001-0001
74 câu lệch vị trí đáp án vài ký tự	Chuẩn hoá khoảng trắng làm dịch vị trí	Căn lại theo lần xuất hiện gần nhất nếu lệch ≤ 20 ký tự

Lỗi phát hiện	Nguyên nhân	Cách xử lý
transformers 5.x không tải được tokenizer ViT5	Lỗi chuyển đổi spiece.model	Ghim transformers 4.57.6

Bảng 3.2: Các lỗi dữ liệu đã phát hiện và sửa

3.4. Thống kê độ dài và lựa chọn tham số

Độ dài được đo bằng tokenizer của ViT5 (Bảng 3.3). Đầu vào gồm câu hỏi và một đoạn văn có p99 là 516 token, nên giới hạn **512 token** chỉ cắt 1,0% mẫu. Đáp án có p99 là 59 token, nên giới hạn đầu ra là **64 token** (nếu đặt 32 sẽ cắt 8,1% đáp án). Ghép 3 đoạn đã khoảng 631 token và ghép 5 đoạn khoảng 1.033 token, vượt quá 512. Vì vậy Reader **đọc từng đoạn riêng** thay vì ghép.

Thành phần	p50	p90	p99	Lớn nhất	Trung bình
Đoạn văn	201	328	478	895	220
Câu hỏi	16	24	32	71	17
Đáp án	9	30	59	143	13
Đầu vào Reader (câu hỏi + 1 đoạn)	231	365	516	930	251

Bảng 3.3: Độ dài theo token ViT5 (nguồn: reports/data_stats.md)

> **Nhận xét:** Đáp án ViQuAD khá dài (trung vị 9 token, tức khoảng 8 âm tiết). Vì vậy EM khắt khe hơn nhiều so với F1: chỉ cần thừa hoặc thiếu một chữ ở biên là EM bằng 0. Báo cáo dùng F1 làm thước đo chính.

4. PHƯƠNG PHÁP

4.1. Retriever hybrid

Kho chỉ có 1.862 đoạn, nên với mỗi câu hỏi Retriever **chấm điểm toàn bộ kho** thay vì tìm gần đúng. Nhờ vậy RRF gộp được trên thứ hạng đầy đủ của cả hai nhánh.

- **Nhánh BM25** (rank-bm25): tách token theo **âm tiết** ("điện | biên | phủ"), chữ thường, bỏ dấu câu. Thử tách theo từ bằng underthesea nhưng kết quả kém hơn, có thể do tách sai tên riêng nước ngoài.
- **Nhánh dense:** mô hình bkai-foundation-models/vietnamese-bi-encoder. Câu hỏi và đoạn văn được tách từ trước, vì mô hình dựa trên PhoBERT yêu cầu vậy. Độ dài tối đa 256 token (11,8% đoạn bị cắt khi mã hoá). Embedding chuẩn hoá L2 và so bằng cosine trong FAISS.
- **Gộp RRF có trọng số:** điểm(d) = 1/(k + hạng_BM25(d)) + w · 1/(k + hạng_dense(d)), với hạng tính từ 1. Quét k và w trên dev, chọn **k = 5, w = 0,7**. Công thức RRF mặc định (k = 60, w = 1) còn thua BM25 dùng một mình.

4.2. Reader ViT5

4.2.1. Định dạng dữ liệu

Mỗi mẫu là một cặp văn bản → văn bản. Đầu vào có dạng `<q> câu hỏi </q> <ctx> đoạn văn </ctx>` (tối đa 512 token). Đầu ra là văn bản đáp án, hoặc chuỗi "không có câu trả lời" (tối đa 64 token). Cùng một hàm định dạng được dùng cho cả huấn luyện và suy luận.

4.2.2. Hai cách tạo dữ liệu huấn luyện

- **R1 (chỉ đoạn đúng):** mỗi câu hỏi đi kèm đoạn văn gốc của nó. Có 9.394 mẫu, trong đó 32,4% có đích là "không có câu trả lời".
- **R2 (thêm đoạn nhiễu):** ngoài các mẫu của R1, với mỗi câu trả lời được, lấy thêm một đoạn *sai* chọn ngẫu nhiên trong top-5 của retriever và gán đích "không có câu trả lời". Lúc chạy thật Reader phải đọc cả những đoạn retriever tìm nhầm, nên cần học cách nhận ra đoạn không liên quan. Có tổng 15.741 mẫu (58,5% đích từ chối). 183 đoạn nhiễu hoá ra vẫn chứa nguyên văn đáp án nên được giữ đáp án.

4.2.3. Huấn luyện

Cấu hình được chọn trên dev (Bảng 4.1). Sau mỗi epoch, mô hình sinh greedy trên dev (đọc đoạn đúng) và giữ checkpoint có F1 cao nhất. R1 được huấn luyện trên Colab (GPU T4). R2 và các biến thể ở GĐ5 được huấn luyện trên GPU RTX 3060 12 GB với bf16.

Tham số	Giá trị	Ghi chú
Mô hình	VietAI/vit5-base	khởi tạo từ checkpoint huấn luyện trước
Learning rate	1e-4 (AdamW)	3e-4 kém hơn rõ: F1 dev 46,9 so với 57,5
Batch hiệu dụng	32 (8 × tích luỹ 4)	

Tham số	Giá trị	Ghi chú
Số epoch	8	5 epoch cho F1 dev 55,9, F1 vẫn còn tăng ở epoch cuối
Warmup / weight decay	5% / 0,01	
Độ dài đầu vào / đầu ra	512 / 64 token	theo thống kê ở mục 3.4
Chọn checkpoint	F1 dev (đoạn đúng), dừng sớm sau 2 epoch không tăng	
Giải mã	greedy (num_beams = 1)	

Bảng 4.1: Cấu hình huấn luyện Reader ViT5 (R2 chính)

Lưu ý: Lần huấn luyện đầu cho F1 dev vô nghĩa (HasAns-F1 khoảng 1%) dù loss giảm đều. Nguyên nhân: tuỳ chọn group_by_length của transformers 4.57 xáo cả thứ tự tập dev, nên dự đoán bị so với đáp án của câu khác (1.328/1.329 câu lệch). Nhóm đã sửa bằng cách luôn đánh giá dev theo thứ tự gốc, và thêm một bước kiểm tra nhãn trước khi tính điểm. Kết quả lần đầu bị loại bỏ.

4.2.4. Đọc nhiều đoạn và chọn đáp án

Với top-k đoạn, Reader đọc từng đoạn trong cùng một batch. Mỗi đoạn cho một đáp án kèm điểm tin cậy, là log-xác suất trung bình mỗi token sinh ra. Hệ thống chọn đáp án có điểm cao nhất **trong các đáp án không phải từ chối**. Hệ thống chỉ từ chối khi mọi đoạn đều cho "không có câu trả lời".

4.3. Ngưỡng từ chối

Khi Reader định từ chối một đoạn, hệ thống cấm token đầu của câu từ chối để buộc mô hình sinh ra đáp án thay thế tốt nhất. Sau đó so hai điểm: chỉ từ chối khi điểm từ chối lớn hơn điểm đáp án thay thế một khoảng δ. Ngưỡng δ được chọn trên dev theo F1 cho từng cấu hình, rồi áp nguyên cho test. Cả hai chế độ (greedy và có ngưỡng) đều được báo cáo.

4.4. Các baseline

Mã	Mô hình	Mục đích
B0	Chọn ngẫu nhiên một cụm danh từ hoặc một năm trong đoạn văn (underthesea)	Mốc sàn
B1	xlm-roberta-base trích xuất kiểu SQuAD 2.0: cửa sổ 384 token, trượt 128; lr 3e-5, 3 epoch; ngưỡng từ chối chọn trên dev	Đối thủ chính (RQ1)
B2	ViT5-base chưa fine-tune (zero-shot)	Chứng minh tác dụng của fine-tune

Bảng 4.2: Các baseline

Nhóm cố ý không dùng các checkpoint "vi-mrc" có sẵn trên mạng, vì chúng có thể đã được huấn luyện trên chính ViQuAD, kể cả phần dữ liệu dùng làm test ở đây.

4.5. Thước đo và kiểm định

EM (khớp hoàn toàn) và **F1** (trùng một phần theo âm tiết) được tính sau khi chuẩn hoá (Bảng 4.3). Câu không trả lời được chấm theo kiểu SQuAD 2.0: được 1 điểm nếu mô hình từ chối, ngược lại 0. Các chỉ số tách riêng gồm **HasAns-F1** (câu trả lời được), **NoAns-acc** (tỉ lệ từ chối đúng) và tỉ lệ *từ chối nhầm* trên câu trả lời được.

Bước chuẩn hoá	Ví dụ	Lý do
Unicode NFC	ễ tổ hợp = ễ dựng sẵn	cùng chữ nhưng khác mã
Chữ thường	LÊ LỢI = lê lợi	
Thống nhất vị trí dấu thanh	hòa = hoà, thủy = thuỷ	kho dùng lẫn hai kiểu (605 và 953 lần)
Bỏ dấu câu, gộp khoảng trắng	“Bình Ngô đại cáo” = bình ngô đại cáo	
Giữ dấu thanh	Hà ≠ Ha	bỏ dấu làm nhiều từ khác nghĩa thành giống nhau

Bảng 4.3: Chuẩn hoá trước khi tính EM/F1

Chỉ chuỗi gốc rỗng hoặc đúng "không có câu trả lời" mới được coi là từ chối. Quy tắc này sửa một lỗi tinh vi: chuỗi rác toàn ký hiệu của ViT5 zero-shot, sau khi bỏ dấu câu, thành rỗng và từng được tính là từ chối đúng (NoAns ảo 62%). Khoảng tin cậy 95% được ước lượng bằng bootstrap [15] với 2.000 lần lấy mẫu lại câu hỏi. Khi so hai hệ thống, dùng bootstrap cặp trên cùng các câu hỏi.

5. THỰC NGHIỆM VÀ KẾT QUẢ

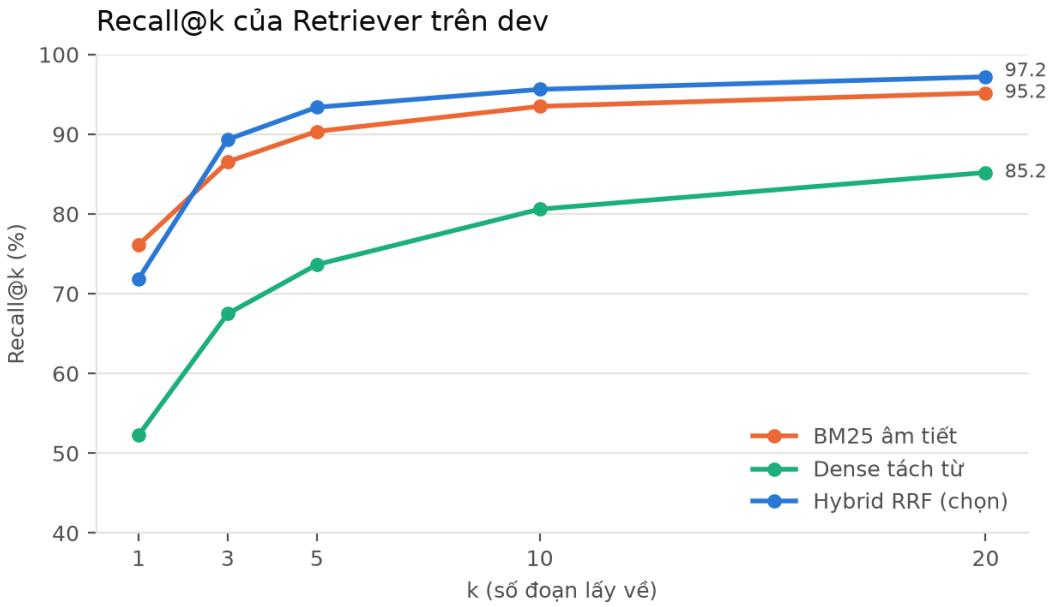
5.1. Thiết lập

Phần cứng: GPU NVIDIA RTX 3060 12 GB, CPU Intel Core i5-14600KF; một phần huấn luyện chạy trên Colab T4. Phần mềm: Python 3.12, PyTorch 2.15, transformers 4.57.6, sentence-transformers 5.7, faiss-cpu 1.15. Mọi dự đoán được ghi vào cùng một định dạng .jsonl kèm commit git, và chấm bằng cùng một công cụ. Nếu không ghi khác, kết quả dùng giải mã greedy. Mã nguồn, dữ liệu đã xử lý và các file kết quả (reports/) có tại https://github.com/trananhhao-0603/hoc_may_tien_tien; README hướng dẫn chạy lại từng bảng.

5.2. Retriever (E2)

Cấu hình	R@1	R@3	R@5	R@10	R@20	MRR
BM25 · âm tiết	76,1	86,5	90,4	93,5	95,2	82,4
BM25 · tách từ	74,8	85,1	88,7	91,5	93,5	80,9
Dense · văn bản thô	49,4	63,7	71,0	76,9	81,6	58,8
Dense · tách từ	52,2	67,5	73,7	80,6	85,2	61,7
Hybrid · BM25 âm tiết + Dense tách từ (k=60, w=1)	67,3	82,1	86,2	92,3	96,7	76,3
Hybrid · BM25 tách từ + Dense tách từ (k=60, w=1)	67,4	80,9	84,8	90,7	95,2	75,5
Hybrid · BM25 âm tiết + Dense tách từ (k=5, w=0,7) (chọn)	71,9	89,3	93,4	95,6	97,2	81,3

Bảng 5.1: Recall@k và MRR của retriever trên dev (892 câu trả lời được)



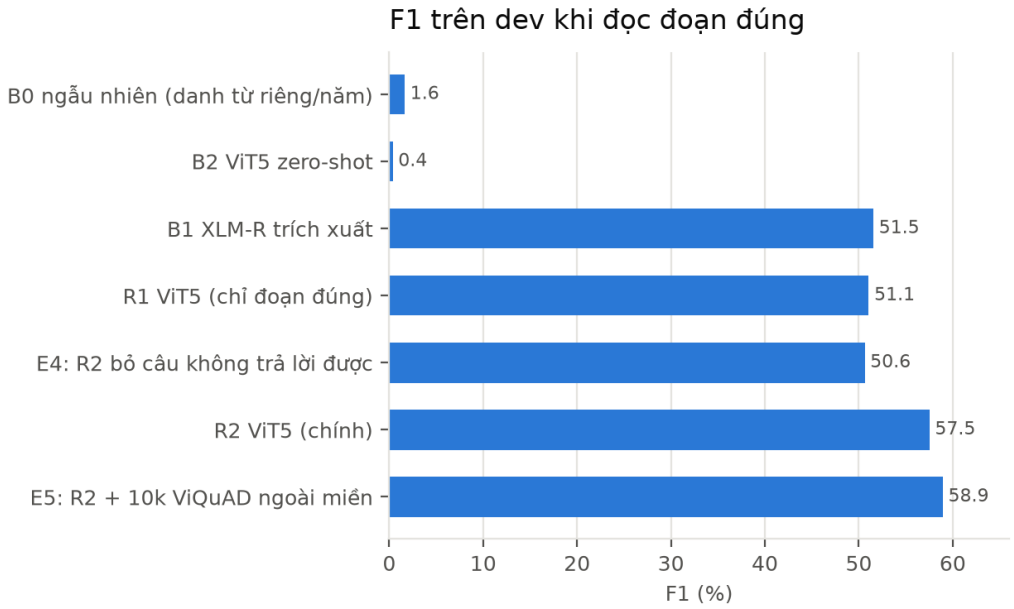
Hình 5.1: Recall@k trên dev của BM25, dense và hybrid đã chọn

Hybrid đạt R@5 = 93,4% trên dev và **90,2% trên test**, vượt mục tiêu 80%. Dense dùng một mình yếu hơn nhiều (R@5 test 71,8%), nhưng bổ sung tốt cho BM25 (86,1%) khi được cho trọng số thấp hơn. R@1 của hybrid thấp hơn BM25. Vì vậy Reader nên đọc nhiều đoạn (k = 3–5) thay vì chỉ một đoạn.

5.3. Reader đọc đoạn văn đúng (E1, E4, E5)

Hệ thống	Tập	EM	F1	HasAns-F1	NoAns-acc	Từ chối nhầm
B0 ngẫu nhiên	dev	0,1	1,6	2,4	0,0	0,0
B2 ViT5 zero-shot	dev	0,0	0,4	0,6	0,0	0,0
B1 XLM-R trích xuất	dev	37,4	51,5	61,7	30,9	20,5
R1 ViT5 (chỉ đoạn đúng)	dev	37,4	51,1	57,3	38,2	27,7
E4: R2 bỏ câu không trả lời được	dev	36,0	50,6	62,2	27,0	18,0
R2 ViT5 (chính)	dev	44,1	57,5	59,2	54,0	23,0
E5: R2 + 10k ViQuAD ngoài miền	dev	44,2	58,9	63,4	49,9	18,6
E4: R2 bỏ câu không trả lời được	test	33,8	51,2	60,9	29,6	16,9
R2 ViT5 (chính)	test	40,4	57,4	58,3	55,4	23,2
E5: R2 + 10k ViQuAD ngoài miền	test	44,6	60,3	60,1	60,7	21,2

Bảng 5.2: Reader khi được cho sẵn đoạn văn đúng (greedy)



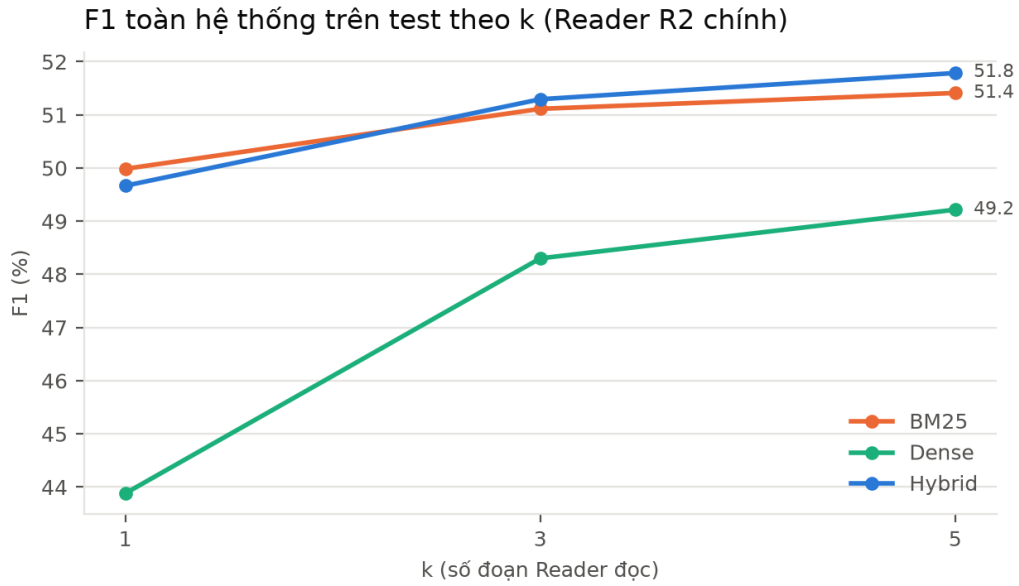
Hình 5.2: F1 trên dev của các Reader khi đọc đoạn đúng

- **E1 (RQ1):** R2 hơn B1 6,0 điểm F1 trên dev (57,5 so với 51,5). Lợi thế nằm gần như hoàn toàn ở khả năng từ chối: NoAns 54,0% so với 30,9%. Trên câu trả lời được, B1 vẫn nhỉnh hơn (HasAns-F1 61,7 so với 59,2). R1 chỉ ngang B1, cho thấy đoạn nhiễu khi huấn luyện là điều kiện để ViT5 vượt B1.
- **Fine-tune là thiết yếu:** ViT5 zero-shot chỉ sinh ký hiệu rác (F1 0,4), thấp hơn cả B0.
- **E4 (RQ3):** bỏ câu không trả lời được khỏi dữ liệu huấn luyện thì mô hình gần như không biết từ chối. Trên test, NoAns giảm 22,8 điểm [−27,6; −17,9] và F1 giảm 5,3 điểm [−7,7; −2,8] (bản có ngưỡng, bootstrap cặp). Đổi lại, HasAns-F1 tăng nhẹ vì mô hình mạnh dạn trả lời hơn.
- **E5 (RQ4):** thêm 10.000 câu ViQuAD ngoài miền lịch sử làm F1 test tăng **+2,9 [+0,7; +5,2]** và EM tăng +4,1 [+1,6; +6,5], cả hai có ý nghĩa thống kê. NoAns tăng +5,3.

5.4. Toàn hệ thống (E2, E3)

Retriever	k = 1	k = 3	k = 5
bm25	50,0	51,1	51,4
dense	43,9	48,3	49,2
hybrid	49,7	51,3	51,8
hybrid + Reader E5	51,9	54,0	53,4

Bảng 5.3: F1 toàn hệ thống trên test theo retriever và số đoạn k (Reader R2 chính, trừ dòng cuối)



Hình 5.3: F1 toàn hệ thống trên test theo k

Số đoạn k được chọn trên dev là k = 5. **Hệ thống cuối** (Reader E5 + hybrid top-5) đạt trên test **EM 39,4 / F1 53,4 / HasAns-F1 52,2 / NoAns 55,9**. Trên test, k = 3 cho F1 cao hơn một chút (54,0), nhưng nhóm giữ k = 5 vì đây là lựa chọn trên dev. BM25 và hybrid gần bằng nhau, cả hai hơn hẳn dense.

E3 — tách lỗi tìm và lỗi đọc (RQ2): đoạn đúng nằm trong top-5 ở 90,2% câu trả lời được. Khi retriever tìm đúng, HasAns-F1 là 57,5, gần bằng mức khi cho sẵn đoạn đúng. Khi tìm sai, HasAns-F1 chỉ còn 4,2. Từ "cho sẵn đoạn đúng" (F1 60,3) sang "tự tìm" mất 6,9 điểm F1. Như vậy phần lớn sai số còn lại nằm ở **khâu đọc**. Mô hình sinh hầu như không bịa: 98,3% câu trả lời xuất hiện nguyên văn trong đoạn đã đọc.

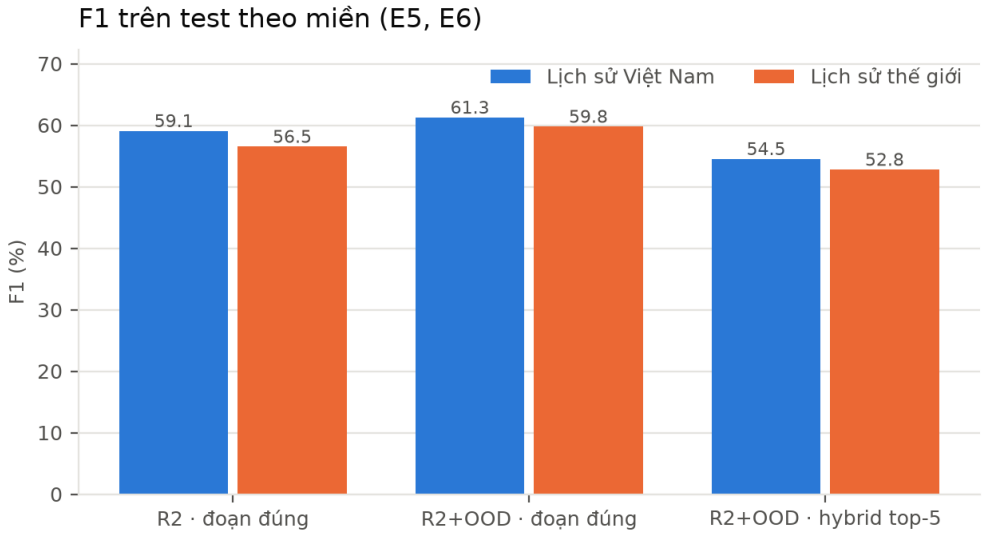
5.5. Ngưỡng từ chối

Trên test, với Reader R2 đọc đoạn đúng, ngưỡng δ chọn trên dev làm NoAns tăng +2,0 [+0,8; +3,4] nhưng HasAns-F1 giảm −1,6 [−2,6; −0,8]. Tổng F1 thay đổi −0,5 [−1,3; +0,2], không có ý nghĩa thống kê. Ngưỡng vì vậy chỉ là núm điều chỉnh giữa "thận trọng" và "mạnh dạn", không phải một cải tiến. Cấu hình mặc định dùng greedy.

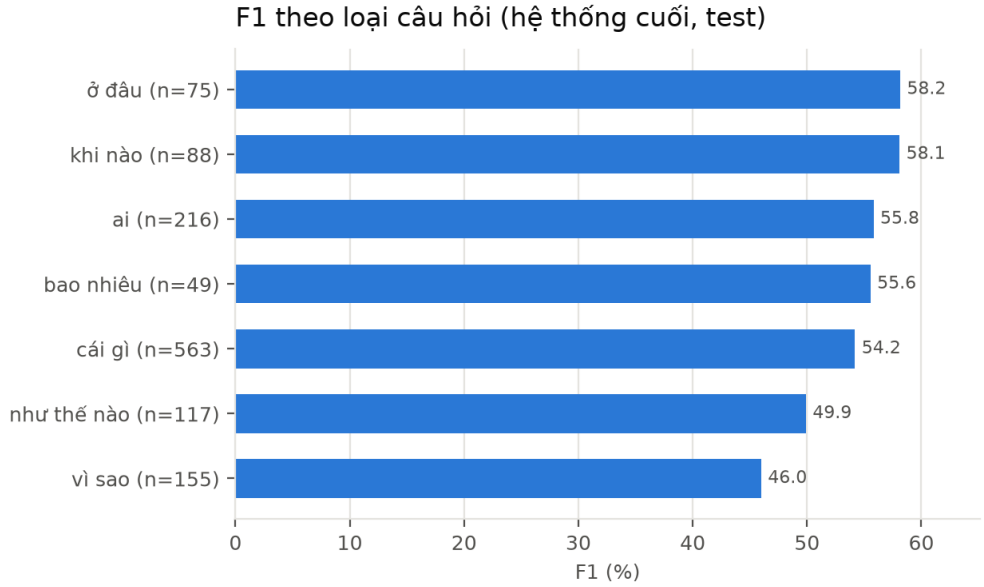
5.6. Theo miền và loại câu hỏi (E6)

Cấu hình (test)	Lịch sử Việt Nam	Lịch sử thế giới
R2 · đoạn đúng	59,1	56,5
R2 + ngoài miền · đoạn đúng	61,3	59,8
R2 + ngoài miền · hybrid top-5	54,5	52,8

Bảng 5.4: F1 theo miền trên test



Hình 5.4: F1 trên test theo miền



Hình 5.5: F1 theo loại câu hỏi của hệ thống cuối trên test

Câu hỏi về lịch sử Việt Nam dễ hơn lịch sử thế giới một chút ở mọi cấu hình. Dữ liệu ngoài miền giúp cả hai miền. Theo loại câu hỏi, câu "ở đâu", "khi nào" và "ai" dễ nhất, vì đáp án ngắn và là thực thể. Câu "vì sao" và "như thế nào" khó nhất, vì đáp án dài và khó xác định ranh giới.

5.7. Độ ổn định

Huấn luyện lại R2 với 3 seed (42, 13, 7). F1 test khi đọc đoạn đúng là **57,3 ± 0,5**, hybrid top-5 là **51,6 ± 0,9**. NoAns dao động nhiều hơn (± 5). Các chênh lệch dưới khoảng 1 điểm F1 vì vậy nằm trong nhiễu giữa các lần huấn luyện. Báo cáo dựa vào khoảng tin cậy bootstrap thay vì so điểm đơn lẻ.

5.8. So với mục tiêu đặt ra

Chỉ số (test)	Mục tiêu	Đạt được	Kết luận
Retriever Recall@5	≥ 80	90,2	Đạt
Reader trên đoạn đúng, HasAns-F1	≥ 70	60,1	**Chưa đạt**
Toàn hệ thống, F1	≥ 55	53,4	**Chưa đạt**
NoAns-acc (Reader E5, đoạn đúng)	≥ 60	60,7	Đạt
ViT5 fine-tune so với zero-shot (dev, F1)	cải thiện rõ	0,4 → 57,5	Đạt

Bảng 5.5: So sánh với các chỉ số mục tiêu đặt ra khi lập kế hoạch

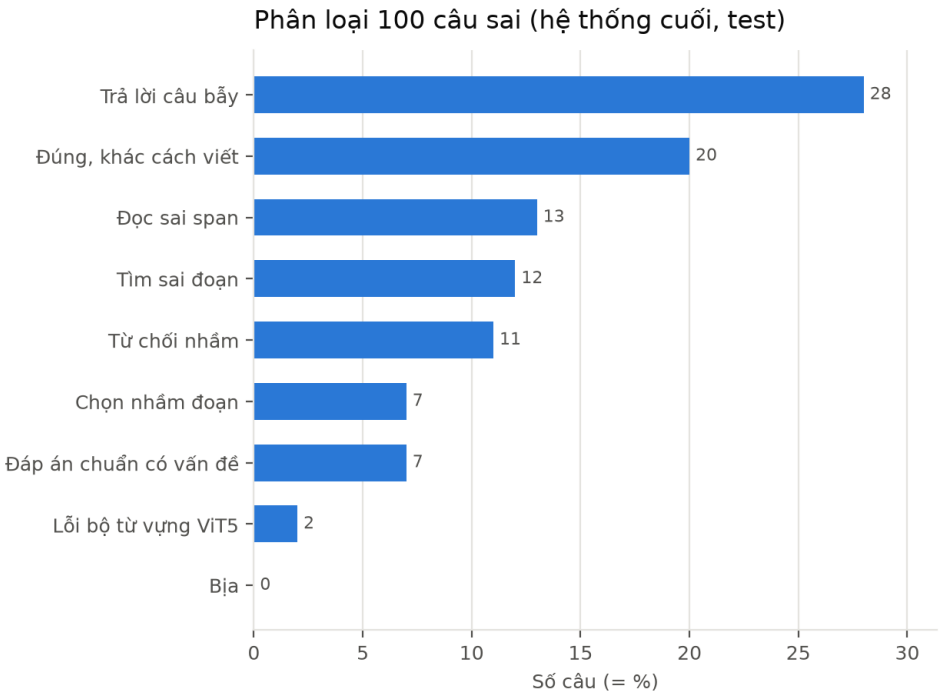
Lưu ý: Các mục tiêu là ước lượng lúc lập kế hoạch. Toàn hệ thống đạt F1 53,4, thấp hơn mục tiêu 55 khoảng 1,6 điểm. Nhóm giữ nguyên con số này và không chấm lại test. Mục 6 phân tích nguyên nhân, và một hướng cải tiến đã thử trên dev được trình bày ở mục 8.3.

6. PHÂN TÍCH LỖI

Từ 779 câu sai (EM = 0) của hệ thống cuối trên test, lấy ngẫu nhiên 100 câu (seed 0). Với mỗi câu, người chấm đọc câu hỏi, đáp án chuẩn, dự đoán, đoạn văn đúng và đoạn mô hình đã dùng, rồi gán một nhãn chính. Công cụ (`src/eval/error_analysis.py`) gợi ý nhãn từ các cờ tự động; nhãn gợi ý trùng nhãn chấm tay ở 70% số câu.

Nhóm lỗi	Số câu / 100	Ví dụ
Trả lời câu bẫy (câu không trả lời được)	28	"Quân Chiêm dẫn quân **vào** Thăng Long tháng mấy?" (đoạn văn: quân Chiêm **rút về**) → "tháng 12"
Đúng nhưng khác cách viết	20	"năm 1910" so với "1910"; "Hiệp ước Versailles" so với "Versailles"
Đọc sai span (có đoạn đúng)	13	trích câu liền sau câu chứa đáp án; thiếu nửa sau của đáp án dài
Tìm sai đoạn (đoạn đúng ngoài top-5)	12	câu hỏi quá chung: "Khuyết điểm của vua Louis là gì?"
Từ chối nhầm	11	cần suy luận: ông nội của Trần Cảnh = cha của Trần Thừa
Chọn nhầm đoạn (đoạn đúng có trong top-5)	7	đoạn đúng ở hạng 1, nhưng đáp án lấy từ đoạn về nhà Đông Hán
Đáp án chuẩn có vấn đề	7	câu gắn nhãn không trả lời được nhưng đoạn văn có đáp án
Lỗi bộ từ vựng ViT5	2	"François II" → "Franois II"; "Mahātmā" → "Mahtm"
Bịa (đáp án không có trong đoạn)	0	không gặp trong mẫu

Bảng 6.1: Phân loại 100 câu sai của hệ thống cuối trên test



Hình 6.1: Phân bố các nhóm lỗi trong 100 câu sai

Nhận xét chính:

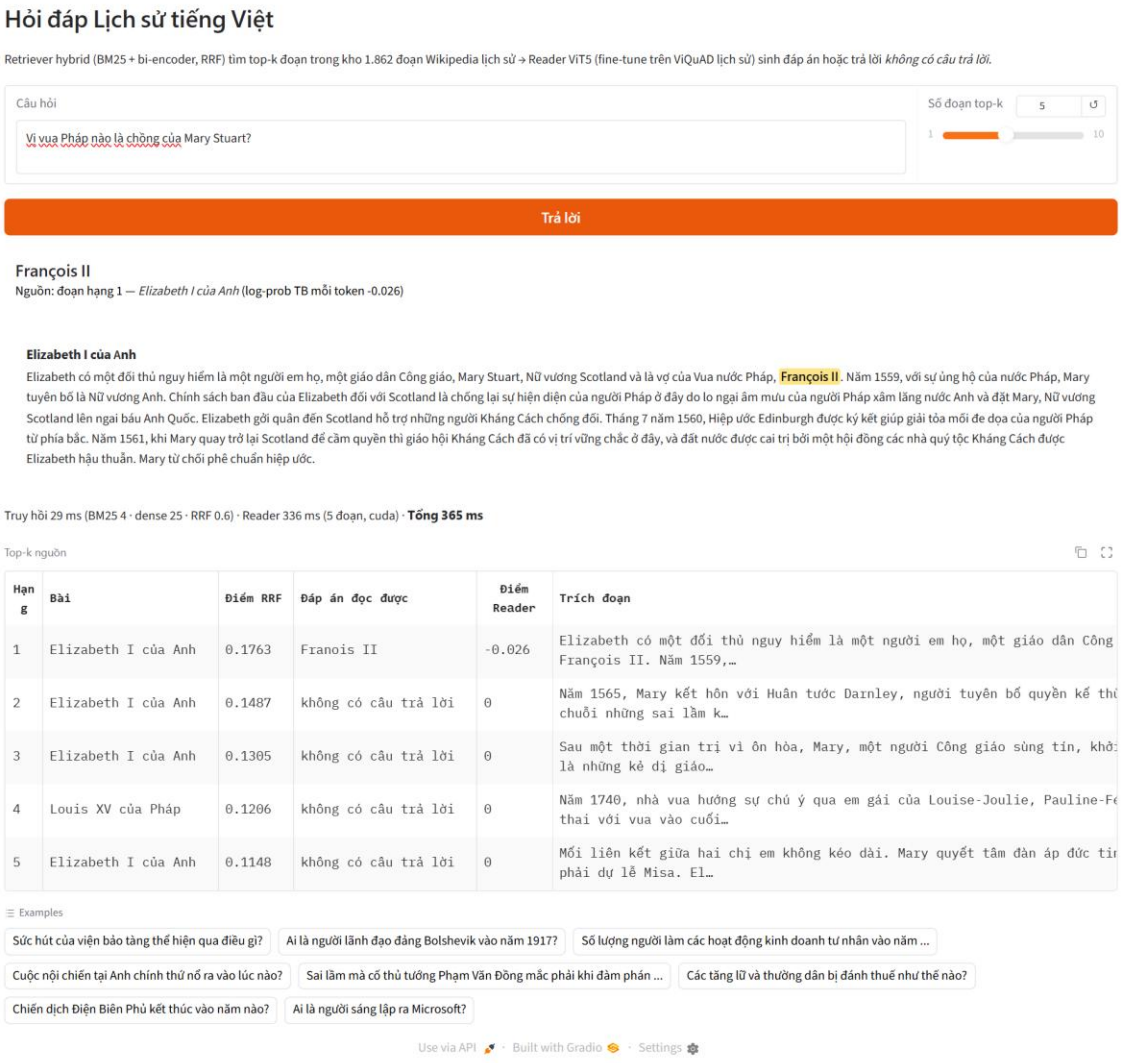
- **Khâu đọc là nút thắt, không phải khâu tìm.** Lỗi do khâu đọc (câu bẫy, span, từ chối nhầm) chiếm 52%. Lỗi do khâu tìm và chọn đoạn chỉ chiếm 19%. Kết quả này khớp với E3.
- **Câu bẫy là điểm yếu lớn nhất (28%).** Câu bẫy của ViQuAD gần trùng đoạn văn, chỉ đổi một chi tiết ("đại biểu" thành "người dân", "hằng năm" thành "hàng tháng"). Reader vẫn trích đáp án mồi.
- **Khoảng 27% "lỗi" thật ra chấp nhận được** (20% khác cách viết, 7% đáp án chuẩn có vấn đề). Độ chính xác thực tế khi người chấm cao hơn EM 39,4. F1 phản ánh tốt hơn EM, nhưng vẫn phạt các đáp án đúng mà dài hơn.
- **Hệ thống gần như không bịa.** Trên toàn bộ 779 câu sai, cờ tự động chỉ thấy 14 câu có đáp án không nằm trong đoạn văn. Trong đó 12 câu là lỗi bộ từ vựng: ViT5 không có các ký tự như ç, ã trong từ vựng nên làm rơi chúng. 9/887 đáp án chuẩn của test (1,0%) không qua được tokenizer nguyên vẹn.
- **Chọn đoạn bằng log-prob trung bình chưa tốt (7%).** Điểm tin cậy của các đoạn khác nhau không so sánh được trực tiếp với nhau.

Lỗi bộ từ vựng sửa được bằng hậu xử lý: tìm trong đoạn văn chuỗi gốc khớp với dự đoán sau khi bỏ các ký tự ngoài từ vựng. Cách này làm F1 test của hệ thống cuối tăng từ 53,37 lên 53,58, nhưng không đổi gì trên dev. Ý tưởng lại nảy ra từ việc đọc lỗi trên test, nên nhóm **không tính đây là cải tiến** và không sửa số liệu ở chương 5. Demo dùng hậu xử lý này để hiển thị đúng tên riêng.

7. DEMO VÀ ĐỘ TRỄ

7.1. Giao diện demo

Demo xây dựng bằng Gradio (`python app.py`), dùng Reader E5 và retriever hybrid. Người dùng nhập câu hỏi và chọn số đoạn k. Giao diện hiển thị đáp án kèm tên bài và hạng của đoạn nguồn, đoạn văn bằng chứng có tô đậm đáp án, bảng top-k đoạn (điểm RRF, đáp án Reader đọc được ở từng đoạn, điểm Reader) và thời gian từng khâu (Hình 7.1). Câu hỏi ngoài phạm vi kho, ví dụ "Ai là người sáng lập ra Microsoft?", thì hệ thống trả lời "không có câu trả lời".



Hình 7.1: Demo Gradio: câu hỏi về chồng của Mary Stuart, đáp án François II được tô đậm trong đoạn bằng chứng

Hình 7.1 cũng minh hoạ lỗi bộ từ vựng ở mục 6. Cột "Đáp án đọc được" là đầu ra thô của ViT5 ("Franois II"); đáp án hiển thị đã được khôi phục từ đoạn văn ("François II").

7.2. Độ trễ

Thiết bị	k	BM25	Dense	Truy hồi	Reader	Tổng
NVIDIA GeForce RTX 3060	1	4	6	11	66	**78**
NVIDIA GeForce RTX 3060	5	4	6	11	118	**128**
Intel Core i5-14600KF	1	5	46	51	208	**250**
Intel Core i5-14600KF	5	5	47	54	969	**1029**

Bảng 7.1: Độ trễ mỗi câu hỏi (ms, trung vị; câu hỏi ngẫu nhiên từ dev)

- **Reader chiếm 80–95% thời gian.** Truy hồi rẻ vì kho nhỏ: BM25 khoảng 5 ms; mã hoá câu hỏi khoảng 6 ms trên GPU và 46 ms trên CPU (gồm cả tách từ).
- Trên CPU, thời gian Reader tăng gần tuyến tính theo k (208 → 969 ms). Trên GPU, 5 đoạn chạy chung một batch nên chỉ tăng gần gấp đôi (66 → 118 ms).
- Cấu hình chính (top-5) mất khoảng 0,13 giây trên GPU và khoảng 1 giây trên CPU, đủ cho sử dụng tương tác.

8. KẾT LUẬN, HẠN CHẾ VÀ HƯỚNG PHÁT TRIỂN

8.1. Kết luận

- **RQ1:** ViT5 sinh đáp án (R2) vượt mô hình trích xuất XLM-R (F1 dev 57,5 so với 51,5), nhờ biết từ chối tốt hơn hẳn. Với câu trả lời được, hai cách gần ngang nhau.
- **RQ2:** Hybrid có trọng số vượt từng phương pháp riêng về Recall@5. Ở cấp toàn hệ thống, hybrid và BM25 ngang nhau và cùng hơn hẳn dense. Sai số chủ yếu đến từ khâu đọc: khi retriever tìm đúng, Reader đạt gần như mức khi được cho sẵn đoạn đúng.
- **RQ3:** Câu không trả lời được trong dữ liệu huấn luyện là thiết yếu. Bỏ chúng đi thì NoAns giảm 22,8 điểm. Đoạn nhiễu do retriever tìm ra (R2) giúp Reader bền vững khi đọc nhiều đoạn.
- **RQ4:** Dữ liệu ngoài miền giúp tăng F1 (+2,9 khi đọc đoạn đúng, có ý nghĩa thống kê) ở cả hai miền. Lịch sử Việt Nam dễ hơn lịch sử thế giới một chút.
- Hệ thống cuối đạt **F1 53,4** trên test, trả lời mỗi câu trong khoảng 0,13 giây trên GPU, và hầu như không bịa.

8.2. Hạn chế

- **Chưa đạt hai mục tiêu:** F1 toàn hệ thống (53,4 so với 55) và HasAns-F1 khi đọc đoạn đúng (khoảng 60 so với 70).
- **Tập đánh giá nhỏ:** test chỉ có 7 bài và 1.286 câu, nên khoảng tin cậy rộng khoảng ± 2,5 điểm F1. B1 và R1 chưa được chấm trên test vì checkpoint chỉ lưu trên Colab.
- **Nhãn "lịch sử" của bài viết kế thừa từ bước rà tay của dự án trước.** Một vài bài chỉ liên quan một phần tới lịch sử, ví dụ Vườn quốc gia Phong Nha - Kẻ Bàng chiếm 327/1.329 câu của dev.
- **Phân tích lỗi do một người chấm,** chưa có đánh giá độ đồng thuận giữa nhiều người chấm. Mẫu được lấy từ test, sau khi test đã chấm xong.
- **Mô hình:** chỉ dùng ViT5-base; Reader đọc từng đoạn độc lập nên không tổng hợp được thông tin từ nhiều đoạn; đầu vào giới hạn 512 token.

8.3. Hướng phát triển

- **Chọn đáp án có xét hạng retriever:** điểm = điểm Reader − λ·(hạng − 1). Thử trên dev (dùng lại dự đoán sẵn có), cách này làm F1 tăng từ 52,92 lên 54,02 (+1,1), ổn định với mọi $\lambda \geq 0{,}02$. Nhóm **chưa chấm trên test** để giữ nguyên tắc chỉ dùng test một lần.
- **Tăng khả năng nhận diện câu bẫy:** sinh thêm câu bẫy khi huấn luyện, hoặc thêm bước kiểm chứng (verifier) so khớp câu hỏi với câu chứa đáp án.
- **Reader đọc nhiều đoạn cùng lúc** theo kiểu Fusion-in-Decoder [6], hoặc thử ViT5-large.
- **Đánh giá bằng người** cho các câu "khác cách viết", để có độ chính xác thực tế bên cạnh EM/F1.
- **Mở rộng kho tri thức** ra toàn bộ Wikipedia lịch sử tiếng Việt. Khi đó cần chỉ mục FAISS gần đúng thay vì chấm toàn kho.

TÀI LIỆU THAM KHẢO

[1] P. Rajpurkar, R. Jia, P. Liang. Know What You Don't Know: Unanswerable Questions for SQuAD. ACL 2018, tr. 784–789. https://aclanthology.org/P18-2124/

[2] K. Van Nguyen, S. Q. Tran, L. T. Nguyen, T. V. Huynh, S. T. Luu, N. L.-T. Nguyen. VLSP 2021 - ViMRC Challenge: Vietnamese Machine Reading Comprehension. arXiv:2203.11400, 2022 (bộ dữ liệu UIT-ViQuAD 2.0). https://arxiv.org/abs/2203.11400 · Dữ liệu: https://huggingface.co/datasets/taidng/UIT-ViQuAD2.0

[3] D. Chen, A. Fisch, J. Weston, A. Bordes. Reading Wikipedia to Answer Open-Domain Questions. ACL 2017, tr. 1870–1879. https://aclanthology.org/P17-1171/

[4] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, W. Yih. Dense Passage Retrieval for Open-Domain Question Answering. EMNLP 2020, tr. 6769–6781. https://aclanthology.org/2020.emnlp-main.550/

[5] P. Lewis, E. Perez, A. Piktus, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020 (Advances in Neural Information Processing Systems 33). https://arxiv.org/abs/2005.11401

[6] G. Izacard, E. Grave. Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering. EACL 2021, tr. 874–880. https://aclanthology.org/2021.eacl-main.74/

[7] L. Phan, H. Tran, H. Nguyen, T. H. Trinh. ViT5: Pretrained Text-to-Text Transformer for Vietnamese Language Generation. NAACL 2022 Student Research Workshop, tr. 136–142. https://aclanthology.org/2022.naacl-srw.18/

[8] C. Raffel, N. Shazeer, A. Roberts, et al. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. JMLR 21(140):1–67, 2020. https://jmlr.org/papers/v21/20-074.html

[9] S. Robertson, H. Zaragoza. The Probabilistic Relevance Framework: BM25 and Beyond. Foundations and Trends in Information Retrieval 3(4):333–389, 2009. https://doi.org/10.1561/1500000019

[10] G. V. Cormack, C. L. A. Clarke, S. Büttcher. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. SIGIR 2009, tr. 758–759. https://doi.org/10.1145/1571941.1572114

[11] A. Conneau, K. Khandelwal, N. Goyal, et al. Unsupervised Cross-lingual Representation Learning at Scale. ACL 2020, tr. 8440–8451. https://aclanthology.org/2020.acl-main.747/

[12] D. Q. Nguyen, A. T. Nguyen. PhoBERT: Pre-trained Language Models for Vietnamese. Findings of EMNLP 2020, tr. 1037–1042. https://aclanthology.org/2020.findings-emnlp.92/

[13] J. Johnson, M. Douze, H. Jégou. Billion-Scale Similarity Search with GPUs. IEEE Transactions on Big Data 7(3):535–547, 2019. https://doi.org/10.1109/TBDATA.2019.2921572

[14] N. Q. Duc, L. H. Son, N. D. Nhan, N. D. N. Minh, L. T. Huong, D. V. Sang. Towards Comprehensive Vietnamese Retrieval-Augmented Generation and Large Language Models. arXiv:2403.01616, 2024 (mô hình bkai-foundation-models/vietnamese-bi-encoder, dựa trên PhoBERT-base-v2). https://arxiv.org/abs/2403.01616 · Mô hình: https://huggingface.co/bkai-foundation-models/vietnamese-bi-encoder

[15] B. Efron, R. J. Tibshirani. An Introduction to the Bootstrap. Chapman & Hall/CRC, 1993. https://doi.org/10.1201/9780429246593